Deliverable 3, Part 4: Comparison Between Testing Approaches

EECS 3311 Software Design | Group 19 | YorkU Lab Equipment Reservation Platform

Seyed Parsa Shoushtari | 219615574

4. Comparison Between the Approaches

This section compares the three testing approaches used in this deliverable: manually written JUnit test cases, automatically generated test cases produced by Randoop, and AI-assisted test cases generated using ChatGPT. The comparison is structured around four dimensions: code coverage, mutation score, readability, and usefulness.

4a. Code Coverage

Code coverage measures the proportion of executable instructions exercised by a test suite. The manually written test suite achieved the strongest and most targeted coverage across the non-GUI classes. Manual tests were authored with direct knowledge of the project structure and class responsibilities, allowing the team to focus on meaningful methods and realistic object states. The Eclipse coverage report confirms that the manual suite reached 100% instruction coverage for the entire model.Equipment package, including Equipment.java, EquipmentSensor.java, EquipmentStatus.java, EquipmentStatusObserver.java, and UsageRecord.java. Within the model.User package, coverage reached 96.5%, with User.java, UserFactory.java, HeadLabCoordinator.java, and LabManager.java all at or near 100%. The UserController and EquipmentController classes achieved 89.9% and 100% coverage respectively. The overall project-level coverage recorded was 45.4%, which reflects the inclusion of GUI-heavy and persistence-layer classes such as ReservationController (8.4%), PaymentController (0%), model.Payment (0%), and model.Reservation (0%) that were not the primary focus of the manual test suite.

The Randoop-generated suite provided broad automated exploration of method call sequences, but its coverage was less targeted. The JaCoCo report for the Randoop suite recorded 14% overall instruction coverage across the project, with 52% on the controller package, 48% on model.Equipment, 81% on model.User, 59% on model.Payment, 32% on model.Reservation, and 0% on the view package. While Randoop achieved stronger-than-expected coverage on model.User and model.Payment compared to the manual suite, this reflects Randoop's tendency to generate many short, repetitive method call sequences that happen to touch getter and setter paths rather than purposefully testing behavioral logic. Many

generated sequences were redundant or exercised the same paths repeatedly, and Randoop struggled with classes that required initialized state or specific object construction sequences, such as those relying on the Singleton DatabaseManager or the State pattern in Reservation.

The AI-generated suite achieved approximately 30% line coverage across the evaluated classes. ChatGPT was most effective at generating tests for isolated model classes such as Equipment and User, where object creation and field validation are straightforward. It was less effective for controller classes and stateful classes, where the AI made assumptions about object initialization or method availability that did not match the actual implementation. Despite this limitation, the AI-generated suite covered meaningful ground for the classes it could handle and served as a useful supplement to the manual tests.

Approach	Overall Coverage	model.Equipment	model.User	Controllers	Reservation / Payment
Manual (JUnit)	45.4% overall*	100%	96.5%	60-100%	0% (excluded)
Randoop	14% (instructions)	48%	81%	52%	32-59%
AI-Assistant (ChatGPT)	~30% (line)	High	High	Moderate	Low

**The 45.4% overall figure includes GUI and persistence classes that were intentionally excluded from targeted testing. For the non-GUI model and controller classes that were the focus of this deliverable, manual test coverage was substantially higher.*

Overall, the manual approach covered the most meaningful portions of the non-GUI codebase because tests were written with intentional focus. The AI-generated suite was a useful complement for bootstrapping coverage on isolated classes, while Randoop contributed broad but less precise exploration.

4b. Mutation Score

The mutation score measures how effectively a test suite can detect artificially injected faults in the source code. A higher mutation score indicates that the test suite is sensitive to subtle behavioral changes, which reflects the quality and depth of its assertions rather than merely its coverage breadth.

The manually written test suite produced the strongest mutation results, as confirmed by the PIT 1.6.8 report. The tests were designed with an understanding of the intended behavior of each class, and assertions were written to verify specific outputs and state transitions. For example, in UserTestCases.java, the HeadLabCoordinatorTest_approveUser test explicitly checks that a user's status changes from false to true after approval, and that other users in the list remain unaffected. This kind of assertion is precisely the type that kills mutants targeting conditional logic and assignment operations. Similarly, the EquipmentTestCases suite verifies not only that methods execute without error but also that they produce the correct observable state changes, such as confirming that disabling equipment both sets the enabled flag to false and transitions the status to DISABLED simultaneously. These compound assertions are highly effective at catching mutants. The PIT report confirmed a mutation coverage of 99% and test strength of 99% for model.Equipment, and 80% mutation coverage with 89% test strength for model.User, validating the quality of the manual assertions on these classes.

The Randoop-generated suite is expected to produce the weakest mutation score. Randoop generates tests by observing the outputs of random method call sequences and recording them as regression assertions. While this approach captures current behavior, it does not distinguish between correct and incorrect behavior. If a mutant changes a subtle aspect of business logic without causing an observable crash, Randoop tests are unlikely to detect it. The suite also tends to generate many equivalent or redundant test sequences, which inflate the apparent test count without adding mutation-killing power.

The AI-generated suite occupies an intermediate position. Because ChatGPT structured its tests around specific scenarios and included meaningful assertions, it performed better than purely random generation in terms of mutation sensitivity. However, the AI made several incorrect assumptions during generation, some of which had to be corrected manually. Tests that were accepted without modification may still contain assertions that do not precisely capture the intended contract of the method, reducing their ability to kill mutants that affect edge cases.

Approach	Mutation Coverage	Test Strength	Assertion Quality	Fault Detection Depth
Manual (JUnit)	23% overall; 99% model.Equipment; 80% model.User	49% overall; 99% model.Equipment; 89% model.User	High, intent-driven	Strong on business logic
Randoop	Not measured via PIT	Not measured	Low, regression-based	Weak on subtle faults
AI-Assistant (ChatGPT)	Not measured via PIT	Not measured	Moderate, scenario-based	Moderate, prompt-dependent

Mutation coverage and test strength figures are taken from the PIT 1.6.8 report run against the manual test suite in Eclipse. Four test cases that caused compatibility issues with PIT were temporarily commented out during the mutation run; this marginally reduced the mutation coverage figures reported above. The low overall mutation coverage (23%) and test strength (49%) also reflect the inclusion of untested packages such as model.Payment, model.Reservation, and view, which brought down the project-wide average. For the packages that were actively tested, results were strong: model.Equipment achieved 99% mutation coverage with 99% test strength, and model.User achieved 80% mutation coverage with 89% test strength. PIT was not run separately for the Randoop or AI-generated suites.

4c. Readability

Readability refers to how easily a developer can understand, maintain, and extend a test suite without additional documentation. It encompasses naming conventions, test structure, assertion clarity, and logical organization.

The manually written tests were the most readable of the three suites. Test method names followed a consistent and descriptive convention that communicated both the scenario and the expected outcome at a glance. Names such as UserControllerTest_loginUser_wrongPassword and EquipmentControllerTest_markMaintenance make the intent of each test immediately clear. The tests were organized into separate classes by responsibility, with UserTestCases, EquipmentTestCases, and ControllerTest each handling a distinct domain. Within each test, the structure followed a straightforward arrange-act-assert pattern, and assertions were written using specific matchers that reveal the expected

value alongside the actual value. The suite also covered non-trivial scenarios such as duplicate registration, state transitions after removal and re-insertion, and approval workflows, all of which are easy to follow because the setup is minimal and the purpose is explicit.

The AI-generated tests were moderately readable, particularly after manual cleanup. The AI tended to produce tests with descriptive method names that followed a similar convention to the manual suite, such as defaultConstructorSetsStatusAvailable and setHourlyFeeThrowsForNegativeValue. These names are clear and self-documenting. However, the AI-generated tests also exhibited some readability weaknesses in their initial form. Some tests were overly granular, splitting what could be a single well-structured test into many single-assertion methods that test trivially obvious getter and setter behavior. For example, the AI generated separate test methods for each individual getter and setter on the User and Equipment classes, which, while technically correct, adds noise to the test suite and makes it harder to identify which tests cover meaningful behavior. After review and consolidation, readability improved considerably.

The Randoop-generated tests were the least readable of the three suites. Randoop produces test method names such as test001, test002, and so on, with no indication of what scenario is being exercised or what the expected outcome should be. The bodies of Randoop tests often contain long sequences of method calls with intermediate variable assignments, making it difficult to determine the intent of the test or which assertion is the meaningful one. Maintaining or extending Randoop tests requires understanding the full generated sequence, which is impractical without running the test and inspecting its output. The Randoop suite provides essentially no self-documentation.

Dimension	Manual	Randoop	AI-Assistant
Method naming	Descriptive, scenario-based	Generic (test001, test002)	Descriptive, field-level
Test organization	By class and responsibility	Single flat class	By class, somewhat granular
Assertion clarity	Clear, intent-driven	Regression-only, opaque	Clear after cleanup
Maintainability	High	Low	Moderate
Overall readability	1st	3rd	2nd

4d. Usefulness

Usefulness captures the practical value of a test suite in detecting real faults, covering meaningful scenarios, and providing assertions that go beyond trivial execution checks. A useful test suite is one that a development team would rely on to confidently refactor, extend, or debug the system.

The manually written tests were the most practically useful. They exercised realistic workflows that mirror how the system is actually used, such as registering users with duplicate emails, logging in with incorrect credentials, adding equipment with duplicate identifiers, and transitioning equipment through its full status lifecycle. The test for HeadLabCoordinator.approveUser verifies not only that the targeted user is approved but also that other users in the system remain unaffected, which is a meaningful behavioral assertion that could catch regression bugs introduced by incorrect list iteration. The controller tests verified Singleton behavior, ensuring that only one instance of UserController and EquipmentController exists throughout the application lifecycle, which directly corresponds to a core architectural requirement of the system. These tests provide genuine confidence that the system behaves correctly under realistic conditions.

The AI-generated tests were useful primarily as a starting point and for covering isolated model classes. For Equipment and User, the AI produced tests that exercised constructor variants, validated setter behavior, and confirmed that exception-throwing paths worked correctly, such as asserting that passing a null status or a negative hourly fee to Equipment raises an IllegalArgumentException with the correct message. These are meaningful tests that go beyond trivial coverage. However, the AI-generated suite did not exercise multi-object interactions, stateful workflows, or integration between components, which limits its practical value for detecting the kinds of faults that arise in real system usage. The suite also required significant manual review, and several generated tests had to be rejected or rewritten before they were trustworthy.

The Randoop-generated tests were the least useful in practice. While Randoop can occasionally discover unexpected behaviors through random exploration, most of its generated sequences did not correspond to meaningful usage scenarios. The tests served as regression assertions capturing the current state of the system rather than verifying intended behavior, which means they would pass even for incorrect

implementations as long as those implementations are internally consistent. They are also difficult to use for debugging because failing Randoop tests do not communicate which aspect of system behavior is violated. For a project like the Lab Equipment Reservation Platform, where correctness depends heavily on specific business rules and state transitions, Randoop's broad but semantically empty coverage is of limited practical value.

Dimension	Manual	Randoop	AI-Assistant
Realistic scenarios	Yes, mirrors actual usage	No, random sequences	Partial, model-level only
Non-trivial assertions	Yes, state and interaction checks	No, regression snapshots	Partial, field-level checks
Fault detection value	High	Low	Moderate
Multi-object interaction	Yes	Rarely	No
Review effort required	Low	High	Moderate
Overall usefulness	1st	3rd	2nd

Final Summary

Across all four dimensions, the manually written test suite demonstrated the highest overall quality. It achieved the strongest targeted coverage, produced the most meaningful assertions for mutation detection, was the easiest to read and maintain, and provided the most practical value for verifying real system behavior. This superiority is not surprising: manual tests benefit from the developer's understanding of the system requirements and intended behavior, which no automated tool can fully replicate.

The AI-generated suite offered a useful middle ground. It accelerated the initial test generation process for isolated model classes and produced readable, structured tests that required moderate but manageable cleanup. Its coverage and mutation-killing power were lower than the manual suite, but it contributed meaningfully to the overall testing effort and demonstrated that AI assistance can be a practical productivity tool when used with appropriate human oversight.

The Randoop-generated suite was the weakest across all dimensions. Although automated generation is fast and requires no upfront design effort, the resulting tests were difficult to read, provided weak fault detection, and captured behavior rather than verifying intent. Randoop is most valuable as a supplementary exploration tool rather than as a primary testing strategy.

The most effective overall strategy was the combination of all three approaches: using manual tests for correctness and behavioral intent, AI assistance for drafting and accelerating coverage on model classes, and Randoop for broad automated exploration of boundary conditions. No single approach was sufficient on its own, and the interaction between the three suites provided a more complete picture of the system's testability than any one method could have achieved independently.

Dimension	Manual	Randoop	AI-Assistant	Best Approach
Code Coverage (targeted)	45.4% overall; 96-100% on focus classes	14% overall instructions	~30% line coverage	Manual
Mutation Coverage (PIT)	23% overall; 99% model.Equipment; 80% model.User	Not measured	Not measured	Manual
Test Strength (PIT)	49% overall; 99% model.Equipment; 89% model.User	Not measured	Not measured	Manual
Readability	High	Low	Moderate	Manual
Usefulness	High	Low	Moderate	Manual